

# ColorRoom

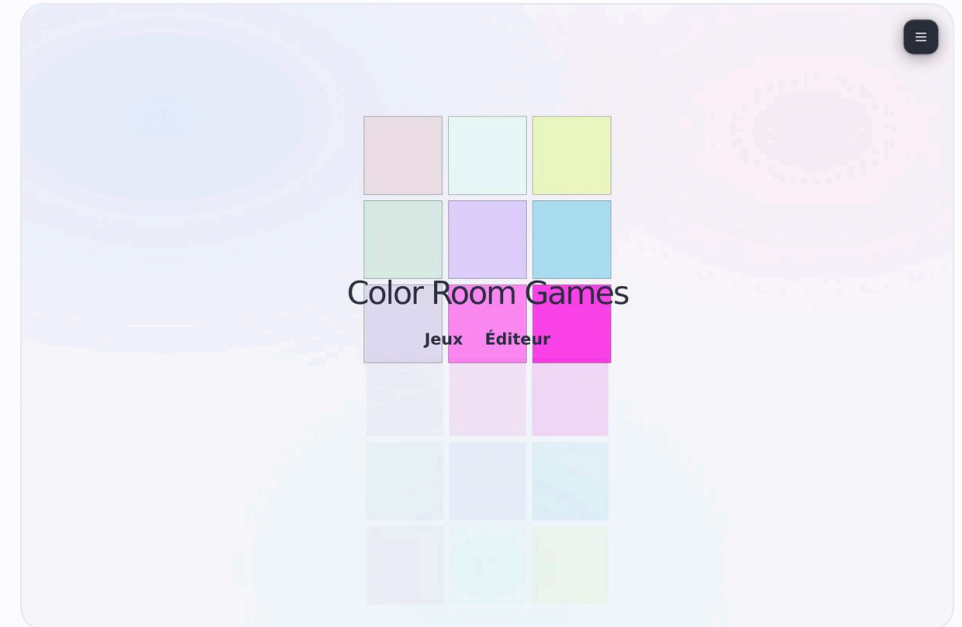
Serious games pédagogiques sur les plaques lumineuses de la ColorRoom

**Téo Trompier** · candidat E2 · sous-équipe JavaScript

Lycée Édouard Branly · Lyon

**LUMEN** – Cité de la Lumière · **ENTPE / LTDS** · labo BPMNP

PILE TECHNIQUE





# Sommaire

---



01 Contexte et commanditaire



03 Besoin et cas d'utilisation



05 Architecture et conception



07 Réseau et déploiement



09 Sécurité et comptes



11 Mesure et chromaticité



02 Le système ColorRoom



04 Équipe et planification



06 Choix techniques et pile



08 Interface et base de données



10 Jeux, IA et multijoueur



12 Qualité, bilan et démo



# Contexte et partenaire

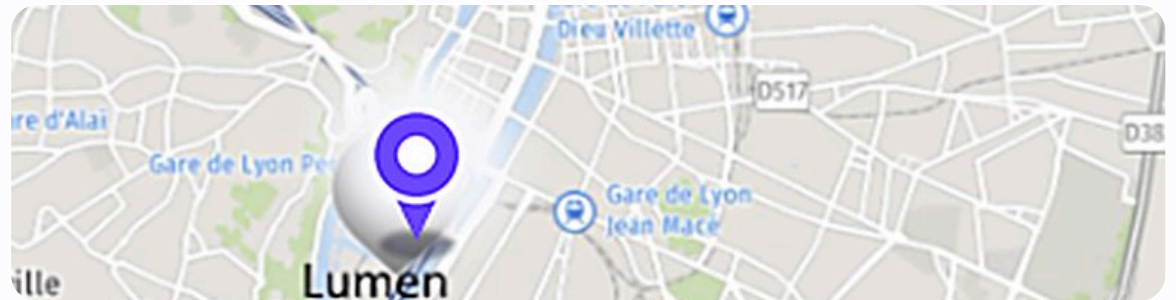
- Commanditaire : laboratoire de recherche **ENTPE / LTDS**, équipe **BPMNP**
- ColorRoom hébergée à **LUMEN – La Cité de la Lumière** (Lyon Confluence)
- Équipement scientifique : **2 cellules jumelles** d'analyse des effets colorés
  - Contacts : M. Labayrade, M. Vella · Professeur : M. Delbosc

**ENTPE** (École Nationale des Travaux Publics de l'État) · **LTDS** (Lab. de Tribologie et Dynamique des Systèmes)

**BPMNP** (Bio-ingénierie, Perception, Mécanique Numérique) · **LUMEN** (Cité de la Lumière)



📍 LUMEN · Cité de la Lumière (Lyon Confluence)



📍 Implantation : LUMEN (Confluence) & ENTPE (agglomération lyonnaise)



## Deux cellules jumelles à visualiser

- **2 cellules jumelles** (2 salles d'analyse identiques)
- Chaque cellule est tapissée de plaques sur les **murs** ET le **plafond**
- **42 plaques** au total (**21 par cellule**) pilotées indépendamment
- Permet de recréer des **ambiances lumineuses** à spectres précis
  - Photo réelle : panneaux allumés sur murs et plafond



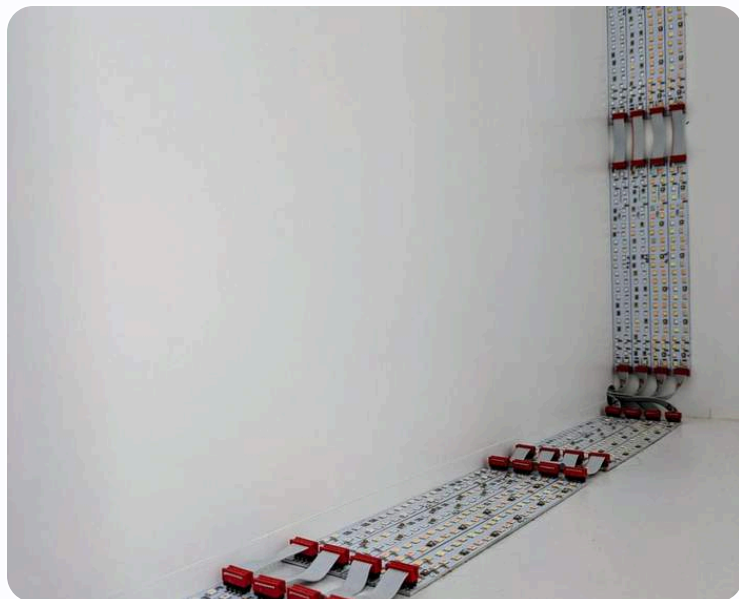
© Laboratoire de la couleur · Campus Lumière





LA PLAQUE LUMINEUSE

# Dimensions et caractéristiques



LED en bandes sur les bords ·  
dimensions

80 × 80  
cm

, épaisseur 23  
~ cm



## 42

plaques (21 par cellule)



## 32

canaux = 24 spectres étroits + 8 blancs



## 2360

LED par plaque (~300 W max)



## 80×80

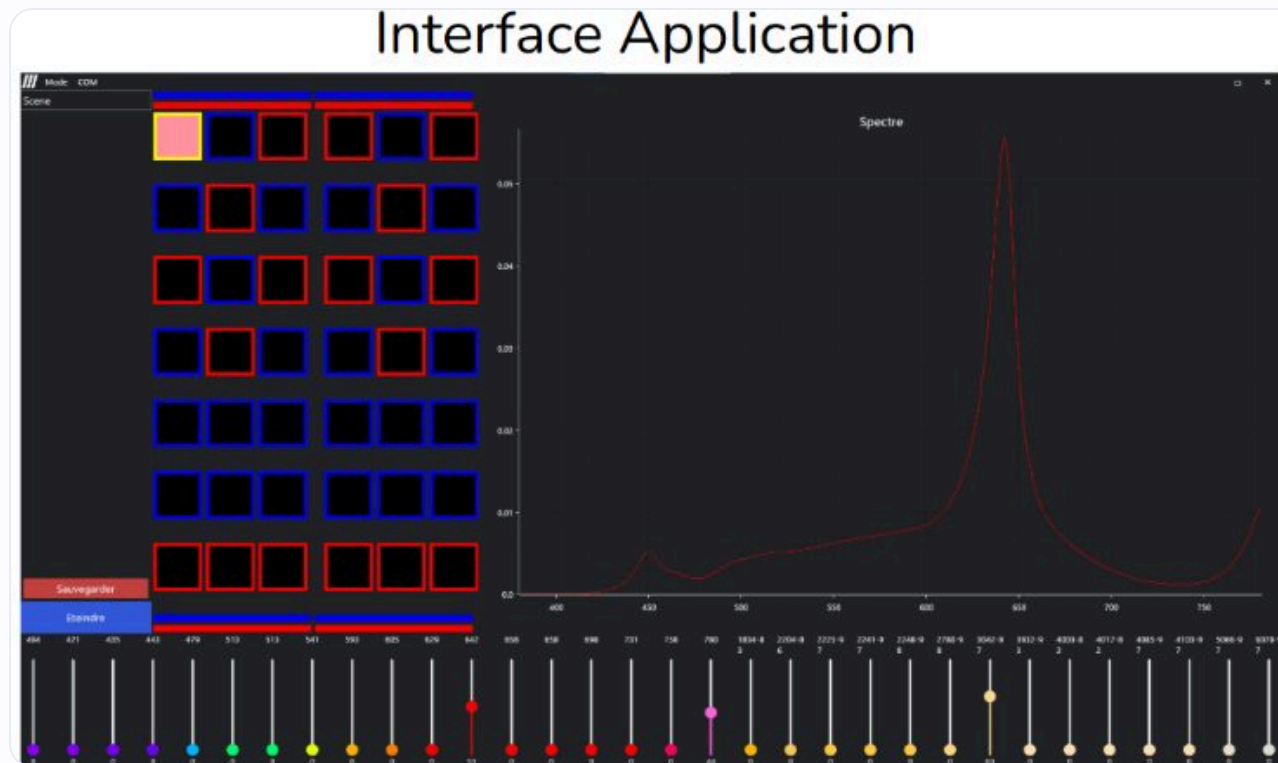
cm par plaque (ép. ~23 cm)

Pilotage en **RS-485** · spectre du **proche UV au proche IR** ; chaque plaque produit couleurs et intensités variées.



# Pourquoi un nouveau logiciel ?

- Logiciel actuel **réservé aux chercheurs** de l'ENTPE
- Pilotage brut : grille des plaques, **32 sliders** de canaux, courbe de **spectre**
- **Trop complexe** pour une initiation : aucune dimension pédagogique
  - D'où ColorRoomGames : une couche **simple et ludique** par-dessus le matériel





# Problématique et objectifs

Le logiciel de recherche de la ColorRoom est trop complexe pour l'initiation. Comment rendre la salle accessible aux apprenants, de façon ludique et pédagogique ?

## 🎯 Objectifs



Une série de **serious games** sur la lumière et la couleur



Accessible depuis un **navigateur**, sans installation



Adaptable à une **seule plaque** lumineuse



## Contraintes



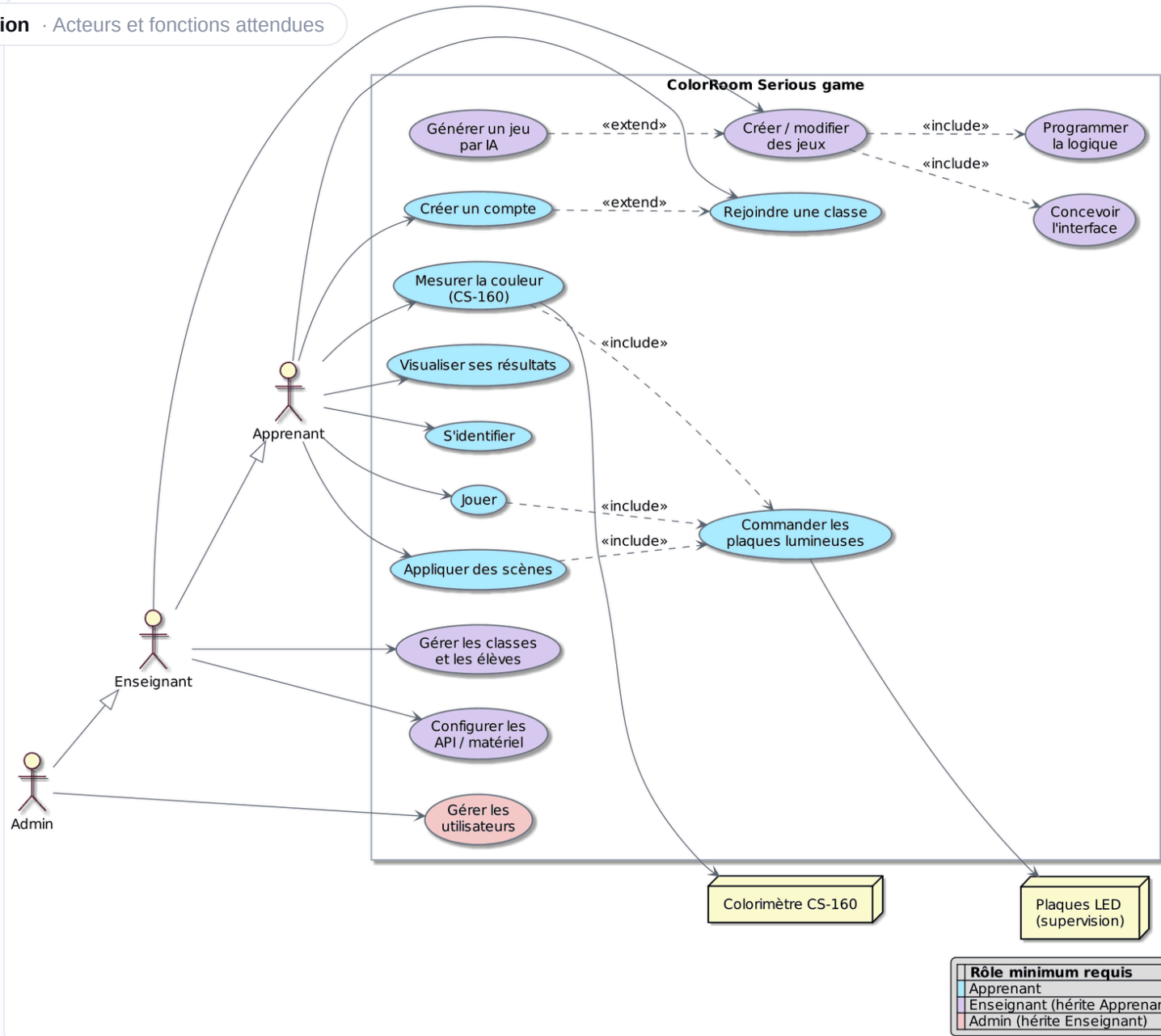
**Raspberry Pi 5** + SSD, Docker imposé



Fonctionnement **hors-ligne** (réseau local)



**Latence** des plaques à compenser







8 ÉTUDIANTS · SOUS-ÉQUIPES JAVASCRIPT ET PYTHON

MA PARTIE · E2

# Équipe et répartition



## Équipe JavaScript · React

E1 → E4 · dont moi (E2)



## Équipe Python · NiceGUI

E5 → E8

MB

### E1 · Maxime Bonnevey

Infrastructure Pi & Docker, CI/CD, intégration colorimètre, sécurité



TT

### E2 · Téo Trompier

MOI

Base de données, API, UI/UX, jeux solo + multijoueur, proxy LED, documentation



IA

### E3 · Ilyes Arbadji

Éditeur de jeux no-code (hors de mon périmètre)

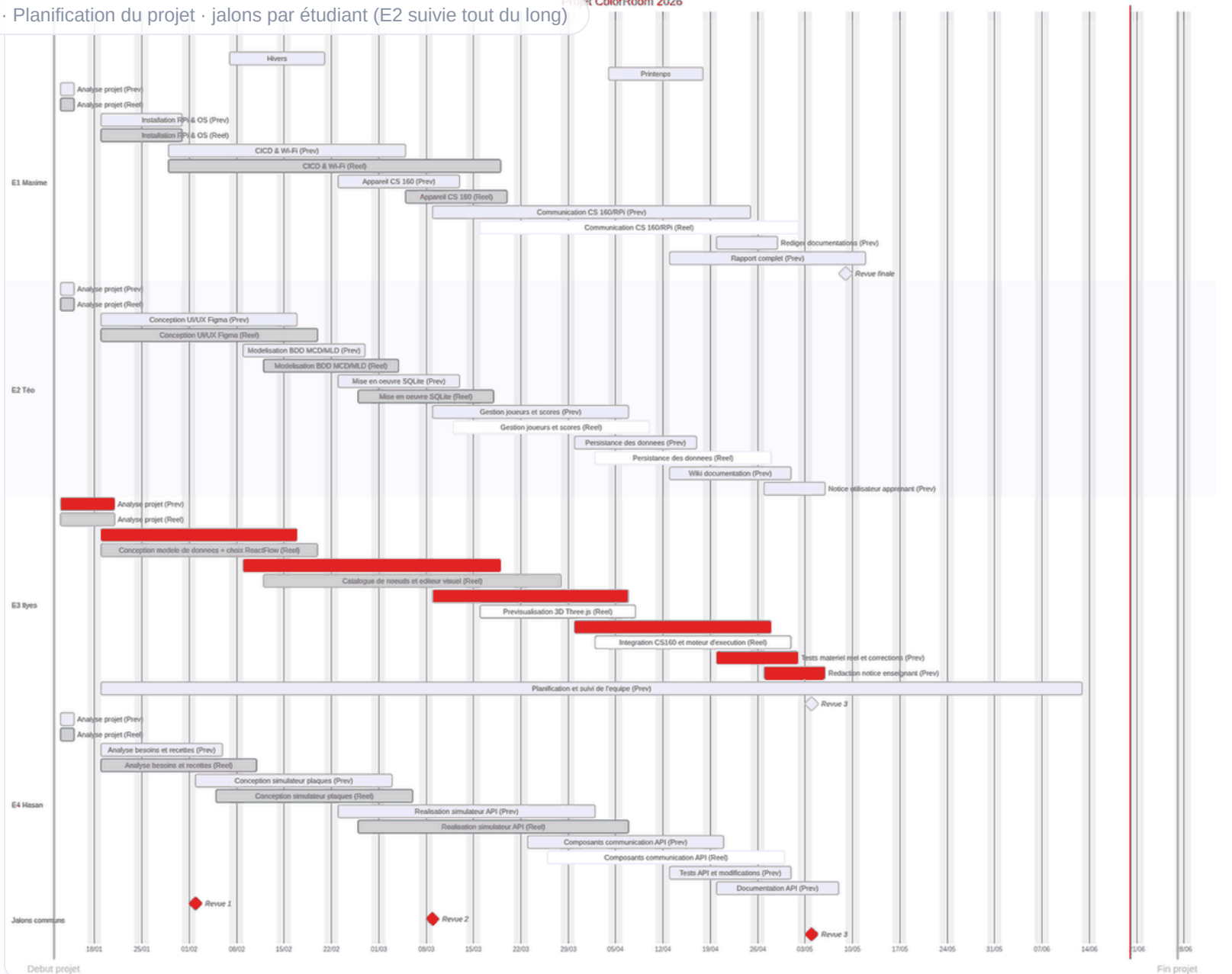


HA

### E4 · Hasan Akyuz

Tests de l'API, simulateur de plaques, Swagger / Postman







# Agile, suivi client et versionnement

## 🎯 Méthode & suivi



Démarche **agile** : développement **incrémental**, livraisons régulières, ajustements



Suivi des tâches via un **tableau Kanban** (À faire / En cours / Terminé)



Échanges réguliers avec le **client M. Labayrade** (directeur du labo **BPMNP**)



**Documentation** du projet rédigée par moi (guide technique, 15 diagrammes UML, notice)

## </> Versionnement



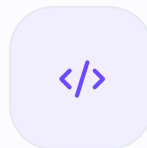
**Git / GitHub** : commits clairs (~280), historique lisible



Travail sur la branche **ux-last** (intégration) puis **merge sur main** (stable)



Chaîne **CI/CD** GitLab → build Docker → déploiement sur le Pi



PARTIE 2 · DE LA PRÉSENTATION GÉNÉRALE À MA CONTRIBUTION

# Ma contribution · E2

Architecture & choix techniques · base de données · sécurité · interface 3D · jeux solo, IA et multijoueur ·  
mesure colorimétrique

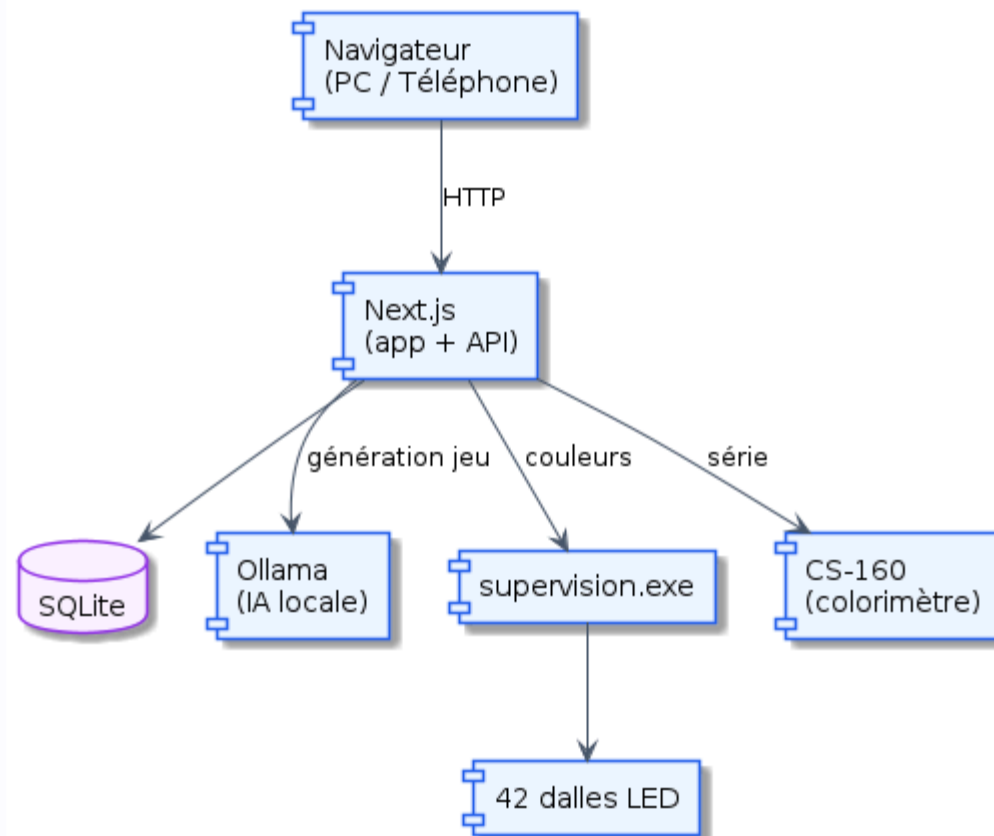




# Architecture logicielle

- Serveur unique **Next.js** (App Router) conteneurisé
- **Route Handlers** + couche **lib/** (auth, db, services)
- Proxy HTTP de **supervision** ; pont **.NET** du CS-160
  - SQLite (better-sqlite3) · 100 % hors-ligne
  - Diagramme **logiciel** : le matériel (Raspberry Pi) figure sur le diagramme de **déploiement**

## ColorRoom - Architecture

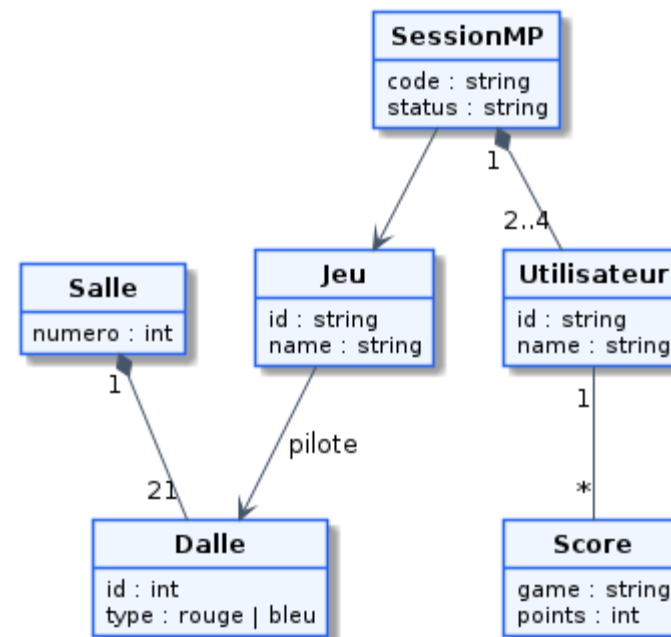




# Diagramme de classes

- Entités métier modélisées et **typées en TypeScript**
- Utilisateur, Classe, Jeu, Score, Session...
- Relations **1-N** (une classe regroupe plusieurs apprenants)
- Vue **objet** du domaine, complémentaire du modèle relationnel

ColorRoom - Classes principales





# Choix techniques · étude comparative

## CRITÈRE

**JS + Node-RED**

PISTE ÉCARTÉE

**Next.js + React + TS**

RETENU

### Façon de programmer

✗ On **relie des boîtes** (flux) : peu adapté à une vraie appli à plusieurs pages

✓ On **assemble des blocs** réutilisables (composants) + pages claires

### Interface

✗ Pas de blocs d'interface réutilisables (3D, catalogue difficiles)

✓ **React** : blocs réutilisables, l'écran se met à jour **tout seul**

### Fiabilité du code

✗ **JavaScript** : les erreurs apparaissent **une fois l'appli lancée**

✓ **TypeScript** : les erreurs sont repérées **avant de lancer** (à l'écriture)

### Maintenance

✗ Fichiers de flux **difficiles à relire** et à suivre dans Git

✓ Code clair, **facile à relire** et à suivre version par version (Git)

### Côté serveur

✗ Logique **dispersée**, liée à l'outil de flux

✓ Front **et back** dans un **seul projet** (Next.js)



# La pile technique



**Next.js**

16.2 · App Router



**React**

19 · composants



**TypeScript**

5.5 · strict



**Node.js**

runtime serveur



**SQLite**

better-sqlite3 11.5



**Three.js**

0.160 · vue 3D



**Docker**

multi-stage arm64



**Raspberry Pi**

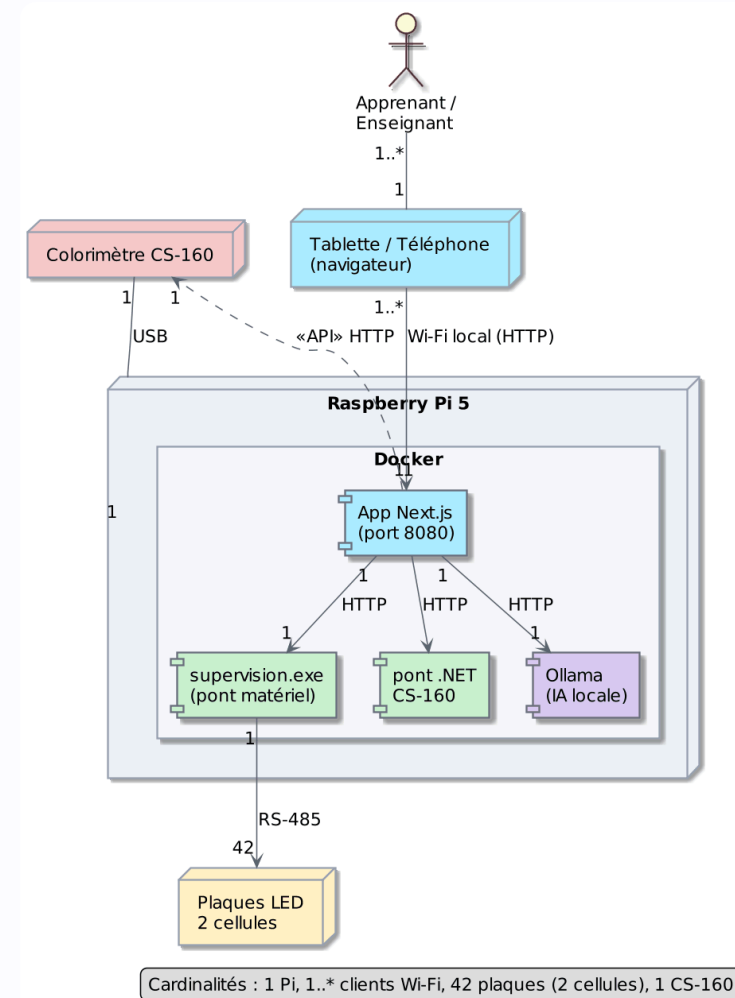
5 · cible





# Réseau et déploiement

- Image **Docker multi-stage** (deps → builder → runner), **arm64**
- App supervisée par **Portainer** · dalles pilotées en **RS-485**
- Wi-Fi local **ColorRoom\_WiFi** ; **CS-160** en USB
- Accès depuis tout appareil : **<http://172.17.40.39/>**
  - SQLite en volume · git pull puis docker compose up





# Adresses des API et canaux LED

- Réglage des **URL d'API à chaud** (Supervision, CS-160) sans redémarrer le serveur
- Stockées en base ; repli sur les **variables d'environnement**
- Bouton **Tester (/health)** pour vérifier la liaison matériel
- Banc de test des **32 canaux LED** (0-255) par dalle, debounce 200 ms

The screenshot shows a web interface for configuring LED channels and API addresses. It includes sections for 'Configuration' (API addresses), 'API Supervision (dalles LED)', 'API CS-160 (colorimètre Konica)', and 'Contrôle des canaux LED (32)'. The 'Contrôle des canaux LED' section features a dropdown for 'Dalle' (set to 'Toutes (1-42)'), buttons for 'Tout au max', 'Éteindre', and 'Envoyer', and a grid of 32 color channels (Ch.0 to Ch.8) with sliders for intensity (0-255).

### Configuration

Adresses des APIs (modifiables à chaud, sans redémarrer le serveur)

Indiquez l'URL complète **avec le port** (ex. `http://172.17.50.136:18080`). Laissez un champ vide pour revenir à la valeur par défaut / variable d'environnement.

#### API Supervision (dalles LED)

source : défaut  
Pilote les 42 dalles. Test : GET /state

`http://172.17.50.136:18080`

#### API CS-160 (colorimètre Konica)

source : défaut  
Bridge de mesure spectrale. Test : GET /api/health

`http://172.17.50.39:3000`

**Enregistrer** **Tester (/health)**

### Contrôle des canaux LED (32)

Dalle : Toutes (1-42) **Tout au max** **Éteindre** **Envoyer**

Contrôlez chaque canal spectral (0-255). Les sliders envoient automatiquement (200 ms de debounce) sur la dalle sélectionnée.

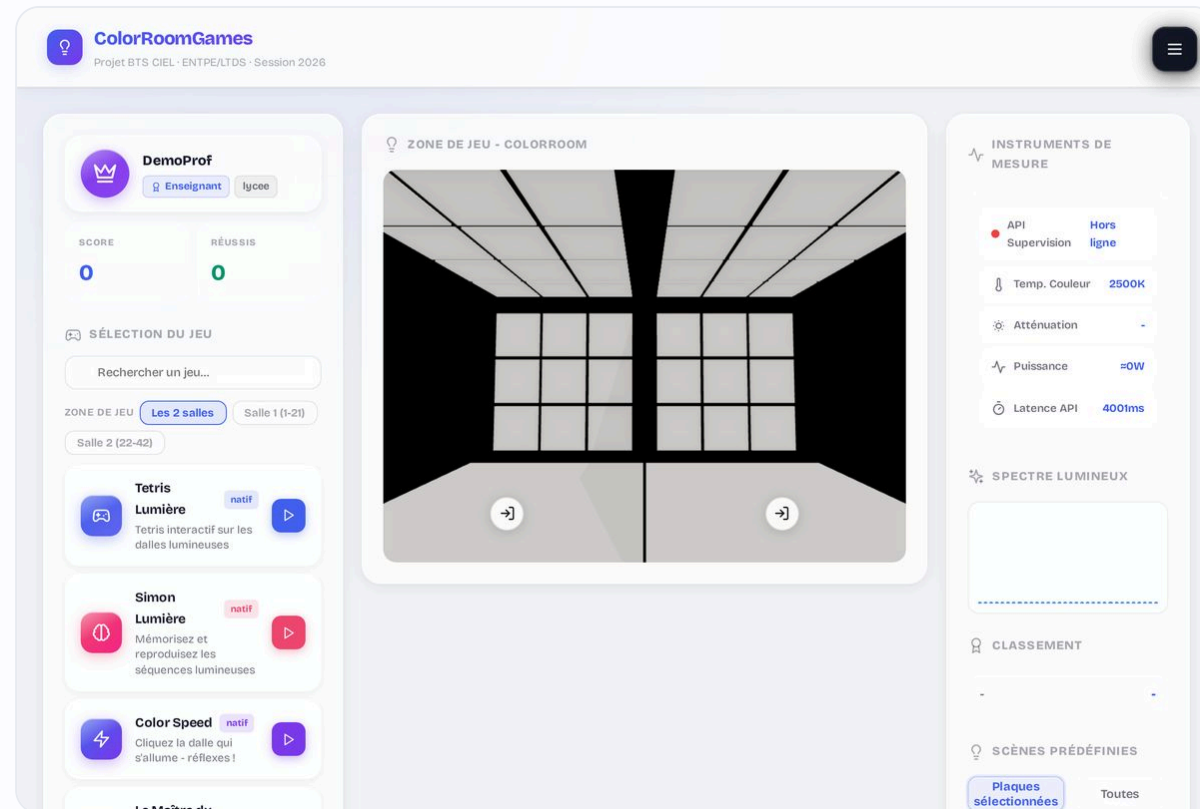
Type de dalle : **Rouge** Bleu

|                     |   |                         |   |                        |   |
|---------------------|---|-------------------------|---|------------------------|---|
| Ch.0 Violet 404nm   | 0 | Ch.1 Violet clair 421nm | 0 | Ch.2 Bleu-violet 435nm | 0 |
| Ch.3 Bleu 443nm     | 0 | Ch.4 Cyan-bleu 479nm    | 0 | Ch.5 Vert 513nm        | 0 |
| Ch.6 Vert (2) 513nm | 0 | Ch.7 Jaune-vert 541nm   | 0 | Ch.8 Orange 593nm      | 0 |



# Interface et design system

- Design system « **verre dépoli** » : variables CSS partagées
- Pages : accueil 3D, /jeux, /mesure, /chromaticite, /gestion, /aide
- **Menu dynamique** selon le rôle
- **Vue 3D** (Three.js : WebGLRenderer, Room3D, 2 cellules)
  - forceContextLoss au démontage · pause via Page Visibility API





# Vue 3D de la salle (Three.js)

- **Scène + caméra + WebGLRenderer** montés dans la page
- Chaque **dalle** = un **Mesh** ; sa couleur suit l'état du jeu en direct
- **Boucle de rendu** via **requestAnimationFrame**
- Au démontage : **forceContextLoss** → évite la fuite de contexte WebGL

</> `app/app/_components/Room3D.tsx`

[github.com/NewGenesisAgency/color\\_room/blob/main/app/app/\\_components/Room3D.tsx#L255-L539](https://github.com/NewGenesisAgency/color_room/blob/main/app/app/_components/Room3D.tsx#L255-L539)

app/app/\_components/Room3D.tsx

```
// Scène, caméra, renderer WebGL
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(CAM_FOV, W/H, 0.05, 80);
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(W, H);
mount.appendChild(renderer.domElement);

// une dalle = un Mesh ajouté à la scène
const plate = new THREE.Mesh(geometry, material);
scene.add(plate);

// boucle de rendu
function loop() { renderer.render(scene, camera); raf = requestAnimationFrame(loop); }
loop();

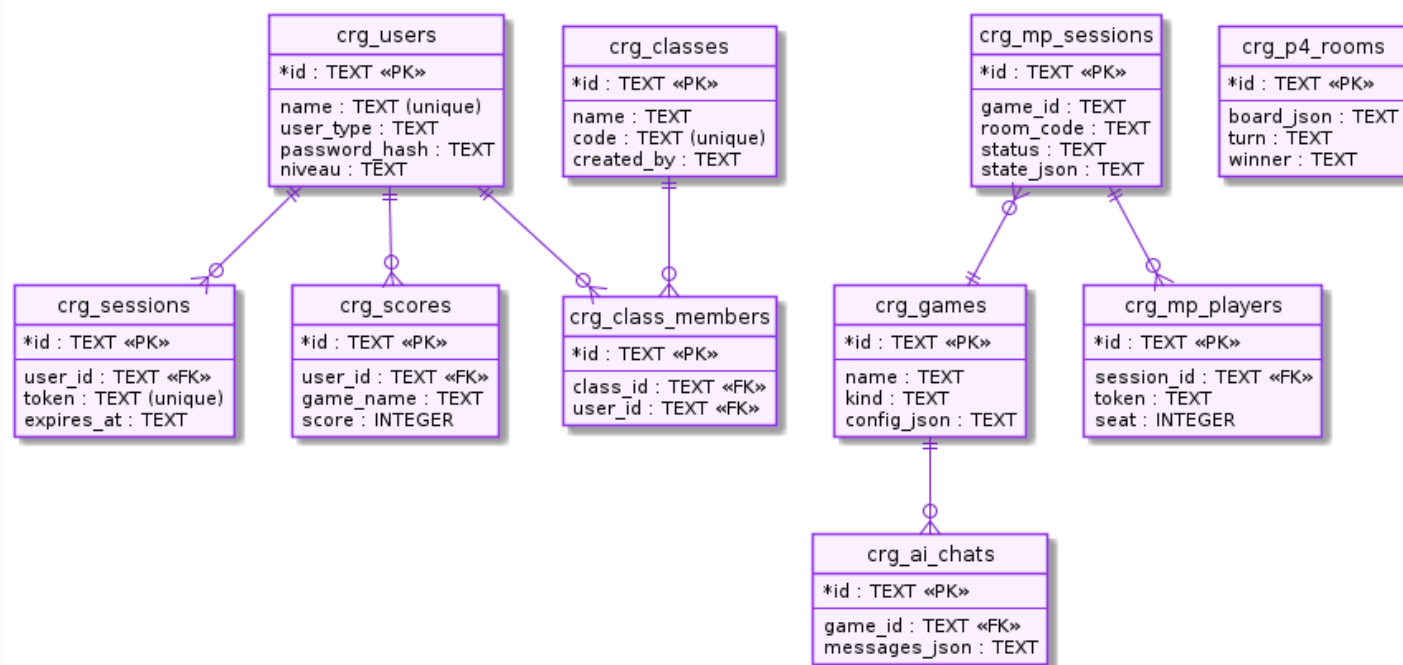
return () => { cancelAnimationFrame(raf); renderer.forceContextLoss(); };
```



# Base de données SQLite

- **better-sqlite3** (API synchrone, requêtes préparées)
- Clés étrangères **ON DELETE CASCADE**
- Migrations **idempotentes** au démarrage
- **journal\_mode=WAL** + busy\_timeout

ColorRoom - Schéma de base de données (SQLite)





# Gestion de l'état

## Persistante

Stockée durablement en SQLite (survit aux redémarrages, volume Docker). `lib/db`

## Transactionnelle · atomique

`db.transaction()` : ACID, tout-ou-rien (user + classe).  
`register`

## Volatile (en mémoire)

`useRef` : état runtime des dalles/jeux, perdu au rechargement. `appl/jeux`

## Environnement

`process.env` : secrets (clé, mot de passe admin) via `.env`, hors Git.

## Réactive

`useState` : déclenche le re-rendu de l'interface à chaque changement.

## Concurrente

Sémaphore `HW_CONCURRENCY=2` : sérialise les accès au matériel. `batch`



# Variable transactionnelle (ACID)

- (ACID = Atomicité, Cohérence, Isolation, Durabilité : les garanties d'une transaction)
- `db.transaction()` encapsule un **BEGIN / COMMIT / ROLLBACK**
- L'inscription = INSERT utilisateur + jonction de classe, en **une seule unité**
- **Atomicité** : exception → **ROLLBACK** total, jamais de compte « à moitié créé »
- Requêtes **préparées** ( `prepare` ) = anti-injection SQL

</> `app/app/api/auth/register/route.ts`

[github.com/NewGenesisAgency/color\\_room/blob/main/app/app/api/auth/register/route.ts#L47-L62](https://github.com/NewGenesisAgency/color_room/blob/main/app/app/api/auth/register/route.ts#L47-L62)

```
app/app/api/auth/register/route.ts

// Variable transactionnelle : tout-ou-rien (ACID).
const insertAll = db.transaction() => {
  db.prepare("INSERT INTO crg_users (id, name,
    user_type, password_hash, ...) VALUES (?,...)")
    .run(id, name, 'apprenant', hash, ...);

  if (classCode?.trim()) { // jonction optionnelle
    const cls = db.prepare("SELECT id FROM
      crg_classes WHERE code = ?").get(code);
    if (cls) db.prepare("INSERT OR IGNORE INTO
      crg_class_members ...").run(rid, cls.id, id);
  }
};

insertAll(); // BEGIN ... COMMIT (ROLLBACK si throw)
```



# Variable atomique · sémaphore matériel

- `hwInFlight` = **compteur atomique** des accès au matériel en cours
- **Atomique** de fait : la boucle d'événements de Node est **mono-thread** (pas d'accès simultané réel)
- Au-delà de **2 slots**, les requêtes attendent dans une **file** (Promises)
- `supervision.exe` est **quasi-série** : on borne pour ne pas le saturer

</> [app/app/api/supervision/batch/route.ts](#)  
[github.com/NewGenesisAgency/color\\_room/blob/main/app/app/api/supervision/batch/route.ts#L39-L80](https://github.com/NewGenesisAgency/color_room/blob/main/app/app/api/supervision/batch/route.ts#L39-L80)

```
app/app/api/supervision/batch/route.ts

const HW_CONCURRENCY = 2; // quasi-série
let hwInFlight = 0;       // variable atomique
const hwWaiters: Waiter[] = []; // file d'attente

async function acquireHwSlot() {
  if (hwInFlight < HW_CONCURRENCY) {
    hwInFlight++; return true; // slot libre
  }
  return new Promise(r => hwWaiters.push({ resolve: r }));
}

function releaseHwSlot() {
  hwInFlight--; // libère un slot
  drainWaiters(); // réveille le suivant
}
```





# État temporaire en mémoire (useRef)

- Une variable **volatile** vit en **mémoire vive** le temps de la partie : **perdue au rechargement**
- **useRef** : valeur mutable **sans re-rendu** (rapide : état de jeu haute fréquence)
- **useState** : volatile **réactif** (re-dessine l'écran quand ça change)
- On ne **persiste** (SQLite) que le **résultat final** : le score

</> [app/app/\\_components/GameColorSpeed.tsx](#)

[github.com/NewGenesisAgency/color\\_room/blob/main/app/app/\\_components/GameColorSpeed.tsx#L160-L181](https://github.com/NewGenesisAgency/color_room/blob/main/app/app/_components/GameColorSpeed.tsx#L160-L181)

app/app/\_components/GameColorSpeed.tsx

```
// Volatile RÉACTIF : re-affiche l'écran quand ça change
const [score, setScore] = useState(0);
const [timeLeft, setTimeLeft] = useState(cfg.duration);

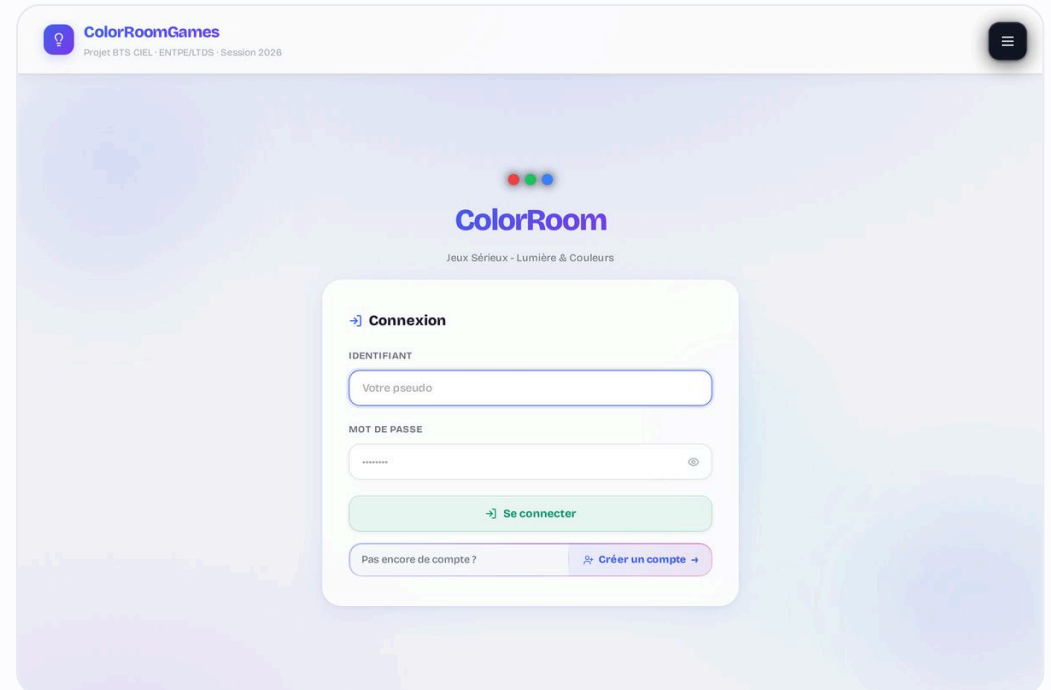
// Volatile PUR : en mémoire, NE déclenche PAS de re-rendu (rapide)
const comboRef = useRef(0); // combo en cours
const lightRef = useRef(0); // dalle allumée
const tileStartRef = useRef(0); // instant d'allumage

comboRef.current++; // mutation directe, sans re-render
// ... en fin de partie SEULEMENT, on persiste le score en base.
```



# Authentification et données personnelles

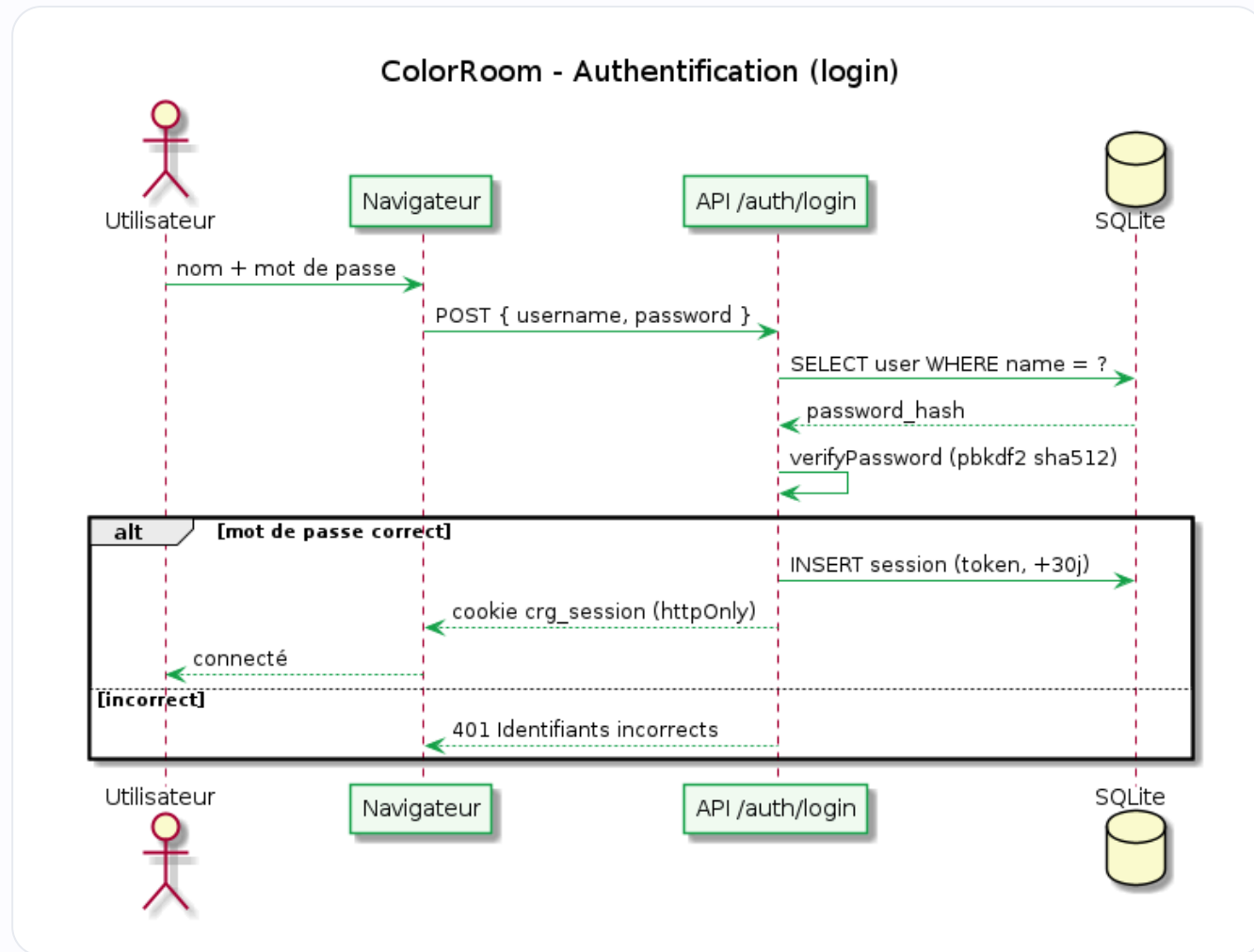
- **3 rôles** : apprenant, enseignant, administrateur
- Les mots de passe ne sont **jamais en clair** : ils sont **hachés** (transformés en empreinte impossible à inverser)
- Connexion gardée par un **cookie sécurisé** (inaccessible aux autres sites)
- **Tout reste en local** : un simple pseudo suffit (respect du RGPD)





# Authentication (PBKDF2 + cookie)

- L'apprenant saisit identifiant + mot de passe
- L'API recalcule le hash via **verifyPassword** (PBKDF2)
- Si valide, création d'une **session** (token aléatoire)
- Renvoi d'un cookie **HttpOnly + SameSite** (30 j glissants)





# Tableau de bord enseignant

- **Classes** : code de 6 caractères (ex. **CS5VHX** , sans 0/O, 1/I)  
+ QR code
- **Suivi des élèves** : niveau et scores par apprenant
- **Gestion des utilisateurs** : rôles, réinitialisation, suppression (admin)
  - Export **CSV** (Blob, BOM UTF-8) · scores en JOIN scores × users

← Jeux **Espace enseignant**  
Vos classes, vos élèves et leurs scores

DemoProf  
ENSEIGNANT

Mes élèves Mes classes Scores

Mes élèves

En tant qu'enseignant, vous voyez uniquement les élèves de vos classes. Vous pouvez réinitialiser leur mot de passe et leur assigner un niveau. La création et la suppression de comptes sont réservées à l'administrateur.

| UTILISATEUR       | RÔLE | NIVEAU | CRÉÉ LE | ACTIONS |
|-------------------|------|--------|---------|---------|
| Aucun utilisateur |      |        |         |         |



# Création de compte et adhésion à une classe

- Inscription guidée en **3 étapes** : pseudo + mot de passe, avatar, classe
- **Rejoindre une classe** en saisissant son **code** (fourni par l'enseignant)
- Création **atomique** (compte + adhésion) puis **connexion automatique**
  - RGPD : un simple **pseudo** suffit, aucune donnée nominative

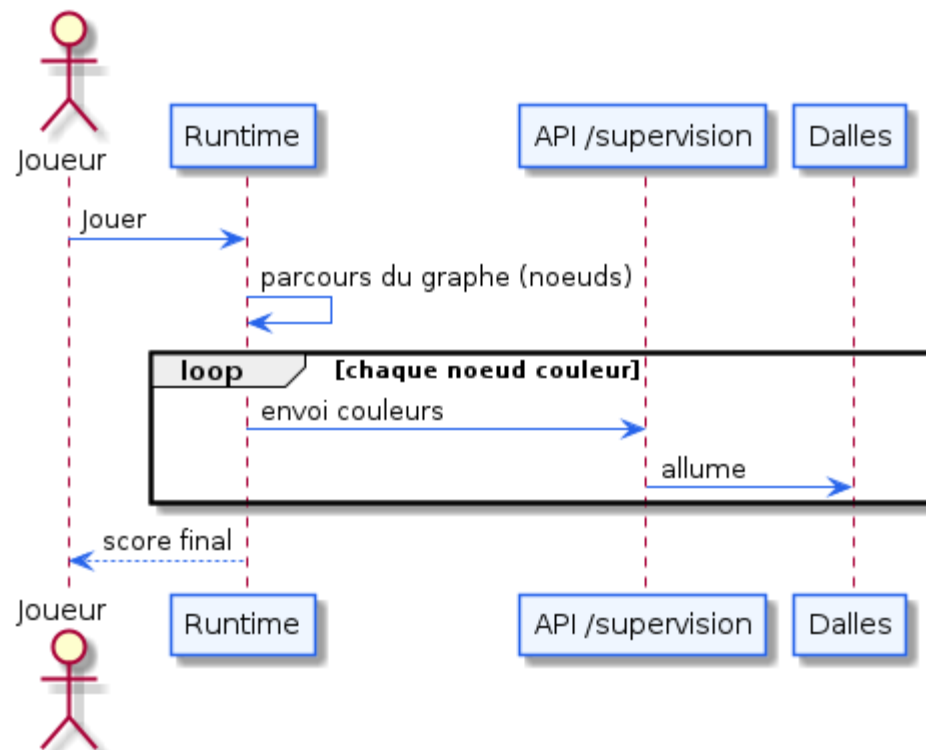
The screenshot shows the 'ColorRoomGames' web application. At the top, the header includes the logo, the text 'ColorRoomGames', and 'Projet BTS CIEL · ENTPE/LTDS · Session 2026'. A hamburger menu icon is in the top right. The main content area has a light blue background with a large, faint 'ColorRoom' logo. In the center, a white card titled 'Créer votre compte' contains the following fields: 'PSEUDO' with a placeholder 'Choisissez un pseudo...', 'MOT DE PASSE' with a placeholder 'Min. 4 caractères' and an eye icon, and 'CONFIRMER' with a placeholder 'Répéter le mot de passe'. A 'Continuer +' button is at the bottom of the card. Above the card, a progress indicator shows three steps: 1 (active), 2, and 3 (Compte).



# Du clic joueur aux dalles

- Le joueur lance une partie depuis le catalogue
- Le **runtime** parcourt le graphe de nœuds du jeu
- Chaque nœud couleur appelle **/api/supervision**
- Les **dalles** s'allument ; le score remonte à l'écran

## ColorRoom - Exécution d'un jeu

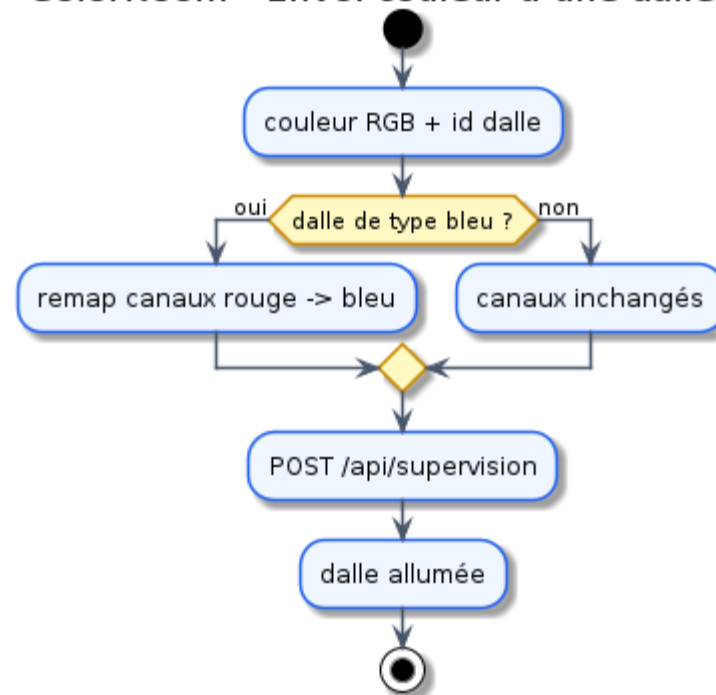




# Remappage des canaux LED

- Entrée : une couleur **RGB** demandée par le jeu
- Conversion vers les **32 canaux** de la plaque
- Application des **profils** (longueur d'onde réelle)
- Envoi de la trame au matériel (proxy supervision)

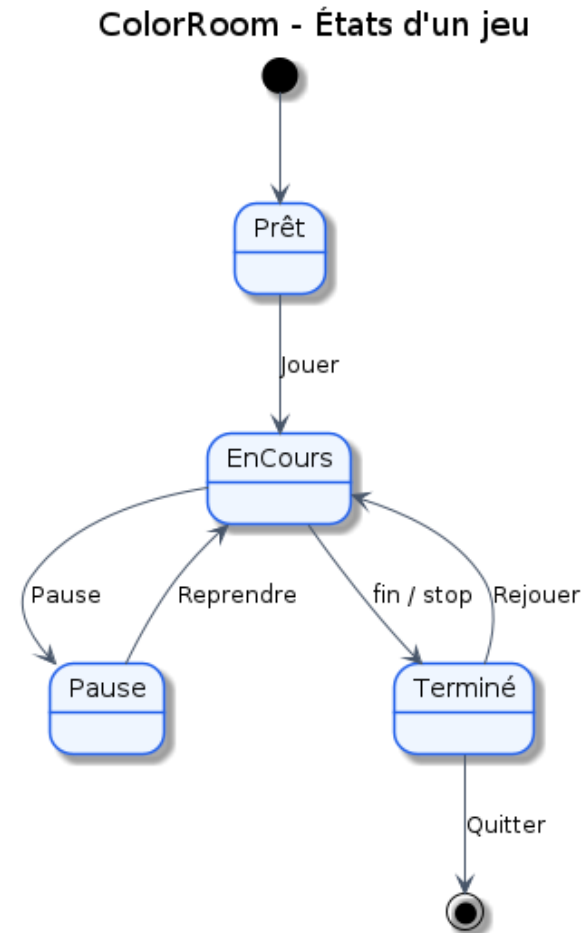
## ColorRoom - Envoi couleur à une dalle





# Cycle de vie d'une partie

- États : **Prête** → **En cours** → **Terminée**
- Transitions : démarrer, jouer un coup, fin de partie
- Calcul et enregistrement du **score** en fin de partie
- Réinitialisation pour **rejouer**

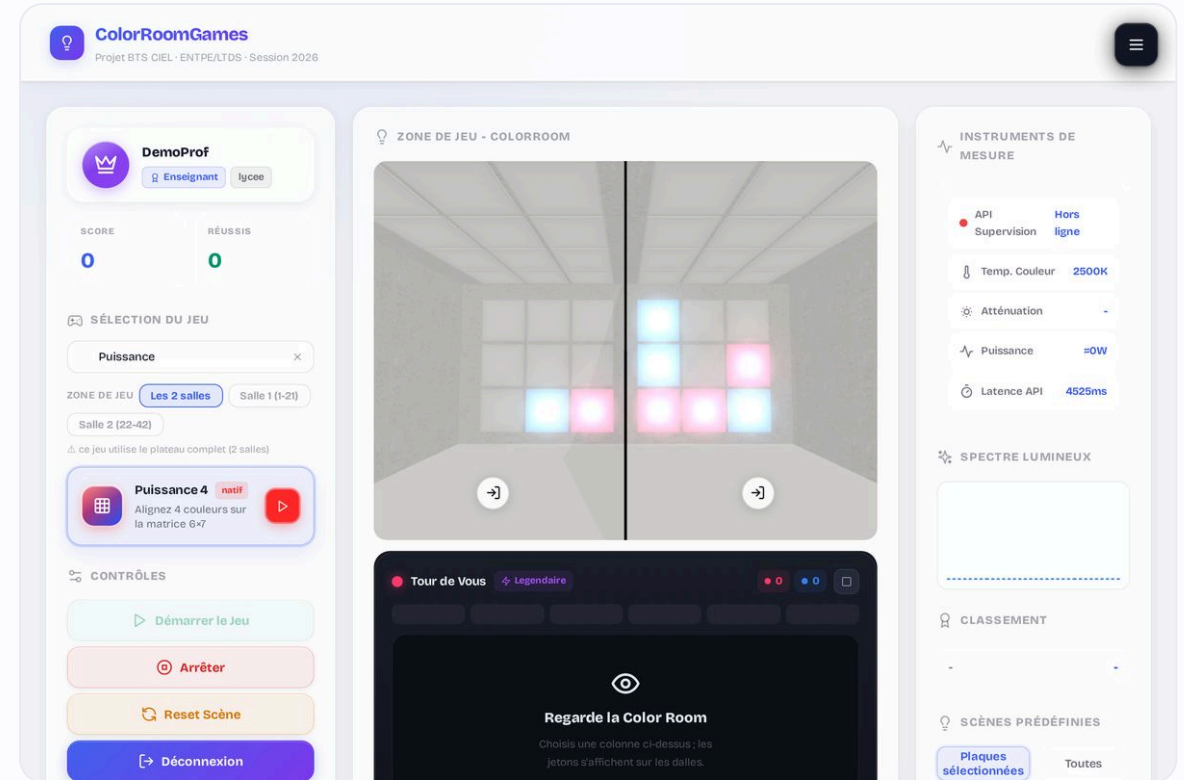






# Puissance 4 et son IA minimax

- Grille 6 colonnes × 7 lignes (42 cases) ; 2 joueurs ou contre l'ordinateur
- IA **hors-ligne** : **minimax** + **élagage alpha-bêta**, anti-piège
- Heuristique par **fenêtres de 4** (défense pondérée > attaque) + poids central
- **5 niveaux** : profondeur **1 / 2 / 5 / 9 / 12** + bruit décroissant





# Évaluation minimax (alpha-bêta)

- Chaque fenêtre de 4 cases est notée du point de vue de l'IA
- **Défense > attaque** : un alignement adverse de 3 vaut -170, le mien +130
- Victoire = **WIN\_SCORE** (1 000 000) ; recherche bornée en profondeur

</> [app/app/\\_components/GamePuissance4.tsx](#)

[github.com/NewGenesisAgency/color\\_room/blob/main/app/app/\\_components/GamePuissance4.tsx#L118-L129](https://github.com/NewGenesisAgency/color_room/blob/main/app/app/_components/GamePuissance4.tsx#L118-L129)

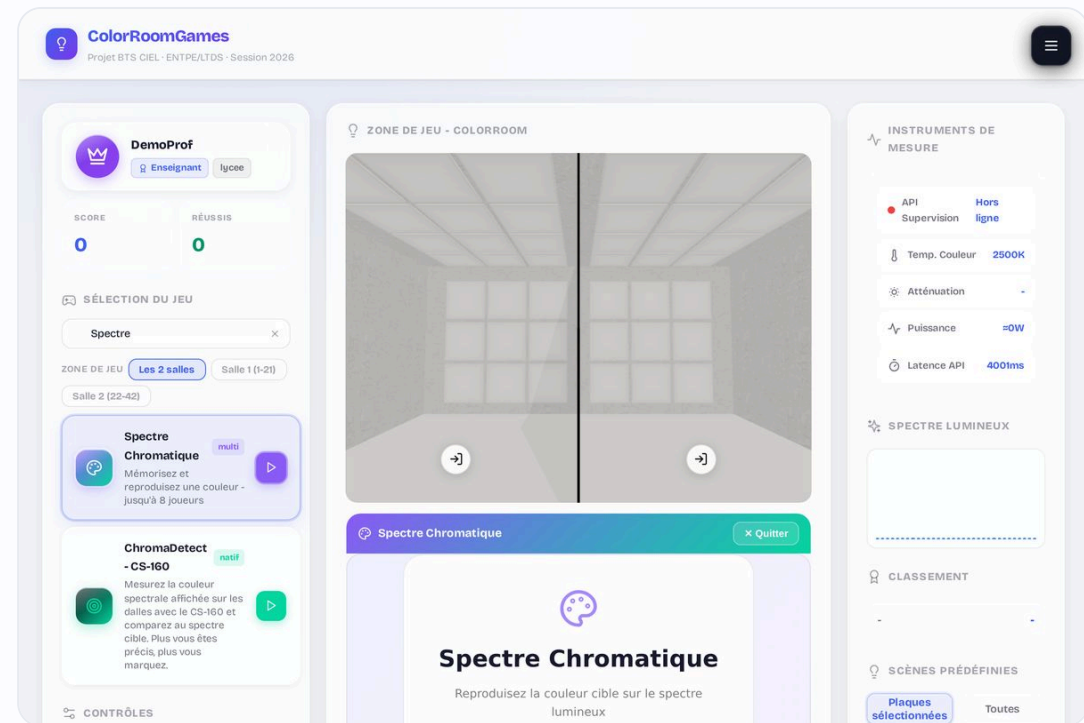
app/app/\_components/GamePuissance4.tsx

```
// Note d'une fenêtre de 4 (défense > attaque)
function scoreWindow(me, opp) {
  if (me>0 && opp>0) return 0; // fenêtre morte
  if (me===4) return WIN_SCORE; // 1 000 000
  if (me===3) return 130;
  if (opp===3) return -170; // bloque la menace
  ...
}
// minimax + alpha-bêta · profondeur 1 → 12
```



# Les jeux multijoueur

- État persisté **en base**, récupéré par **polling** de `/state`
- **Spectre Chromatique** : jusqu'à 8 joueurs (score = distance)
- **Morpion en réseau** : 2 joueurs, détection de victoire
- **Heartbeat** de présence · jetons **UUID** · hôte = siège 1

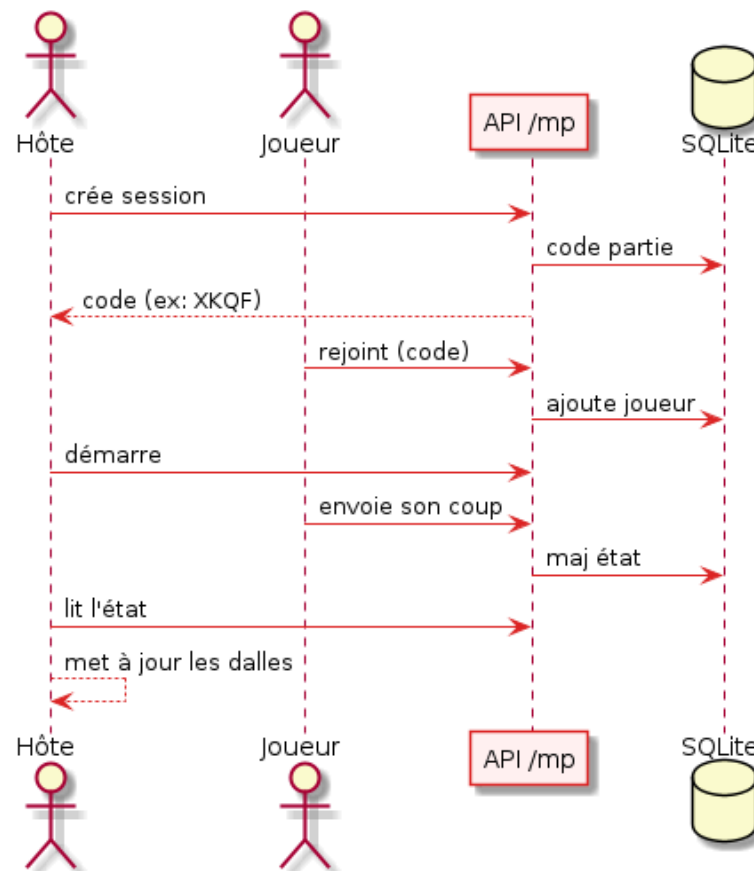




# État partagé et interrogation périodique

- L'hôte crée une session et obtient un **code**
- Les invités rejoignent en saisissant ce code
- L'état est **persisté en base** (state\_json)
- Chaque client interroge **/state** en **polling** → écrans synchronisés

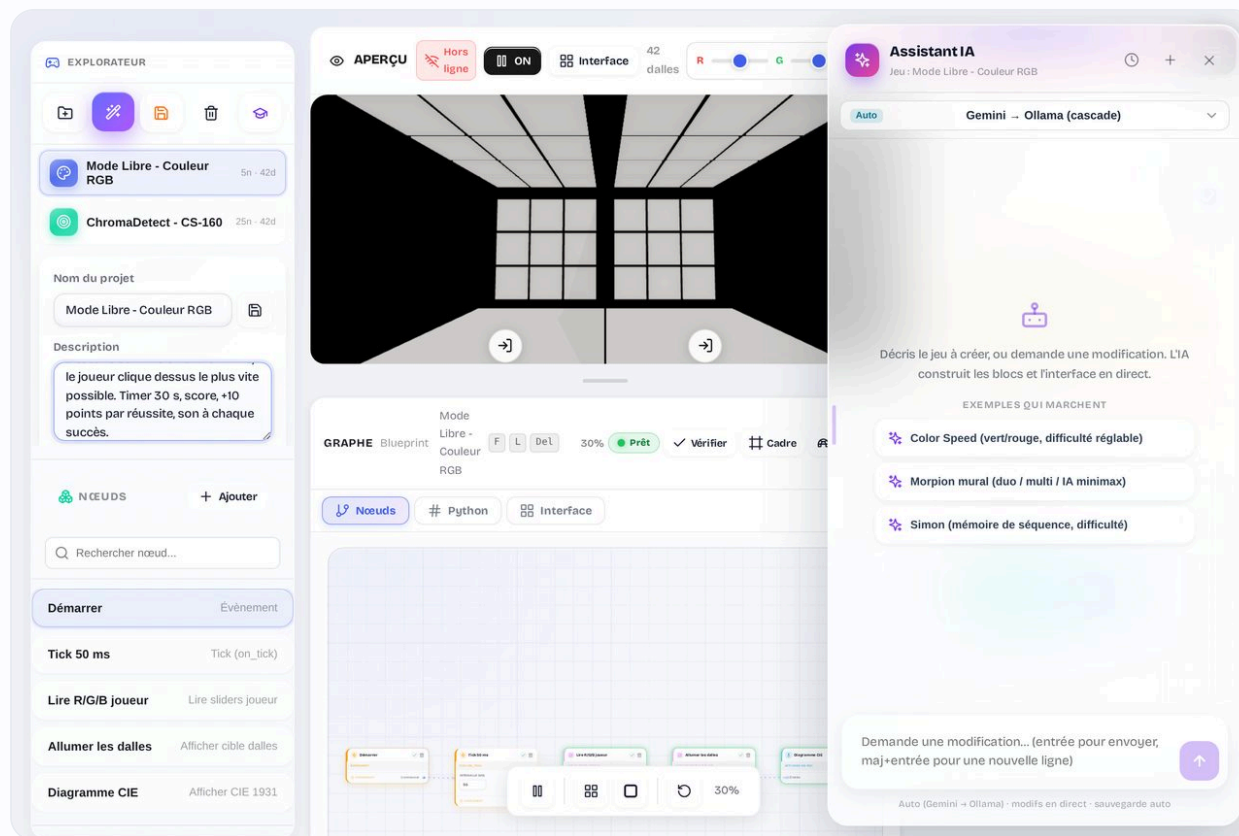
ColorRoom - Session multijoueur





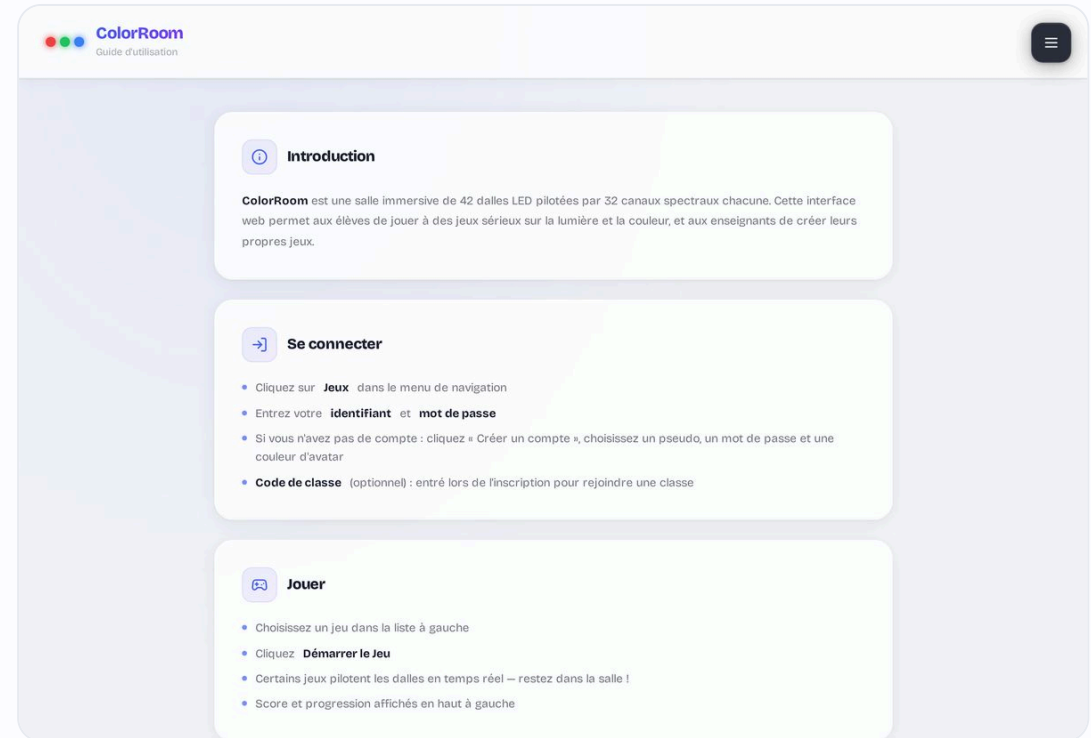
# Créer un jeu sans coder

- Éditeur **no-code** (partie E3) : graphe de **nœuds** + aperçu 3D live
- Mon exploration : « **Créer avec l'IA** » génère un jeu complet depuis une phrase
- Route </api/ai/generate-game> → **Ollama** (local) ou Gemini (cloud, optionnel)
  - Garde-fous : reste **éducatif** ; intègre les nœuds CS-160 et dalles





- Page **/aide** embarquée (hors-ligne) : comment jouer, mesurer, lire le diagramme CIE
- **Documentation** rédigée par moi : guide technique, **15 diagrammes UML**, README d'installation
- Robustesse : **migrations idempotentes**, **transactions ACID**, connexion SQLite en **singleton**
  - Vérif **types (tsc)** + build prod à chaque étape · exports CSV UTF-8 + BOM
  - tests API (E4)





# Difficultés rencontrées et solutions

CORS & pare-feu bloquant l'API Supervision



✓ En-têtes CORS + ouverture du port Windows + portproxy (slide suivante)

Latence des plaques (32 canaux)



✓ Envoi parallèle (Promise.all) + timeout (AbortController)

Couleur écran différente des dalles



✓ Rendu unifié + profils calés sur les longueurs d'onde

Accès concurrents (SQLITE\_BUSY)



✓ Mode WAL + busy\_timeout + connexion singleton

Multijoueur temps réel sur Pi



✓ État en base + polling /state (vs WebSockets)



# Rendre l'API Supervision joignable

- Le navigateur bloquait l'API Supervision (**CORS** + pare-feu)
- L'API écoute en **local** sur **8080** ; on l'expose sur le réseau via **18080**
- À lancer dans **PowerShell (administrateur)** sur le poste hôte de l'API
  - Combiné aux **en-têtes CORS** renvoyés par l'API



Notes d'installation · réseau Windows

PowerShell (administrateur) · poste hôte de la Supervision

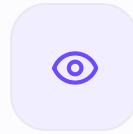
PowerShell · administrateur

```
# 1) Autoriser le port 18080 dans le pare-feu
New-NetFirewallRule -DisplayName "ColorRoom-Supervision" `
  -Direction Inbound -Protocol TCP -LocalPort 18080 -Action Allow

# 2) Rediriger le port 18080 (réseau) vers 8080 (local)
netsh interface portproxy add v4tov4 `
  listenport=18080 listenaddress=0.0.0.0 `
  connectport=8080 connectaddress=127.0.0.1

# L'API écoute en local sur 8080 ; la
# redirection l'expose sur 18080.
```

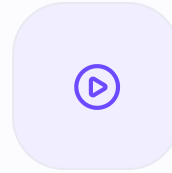




PLACE À LA

# Démonstration

Connexion · catalogue et un jeu sur les dalles · Puissance 4 contre l'IA · un jeu multijoueur · mesure CS-160 et diagramme CIE · tableau de bord enseignant



DÉMONSTRATION

# Vidéo du projet

Présentation filmée de la ColorRoom en fonctionnement · ≈ 2 min

 [video\\_demo.mov](#)